COMPTE RENDU

IncidentDesk

Application de gestion des tickets d'incidents

Réalisé par:

Sabri mohamed Amine

Mohamedi Soulaimane

Rouzana Wiam

Filiere

IA&GI-1

Encadré par:

Mustapha Hain

2026-2027

REMERCIEMENTS

Nous souhaitons adresser nos sincères remerciements à **Monsieur Mustapha Hain**, professeur de Python avancé, pour la qualité de son enseignement, sa disponibilité et ses explications tout au long du cours.

Nous lui sommes particulièrement reconnaissants de nous avoir initié à l'utilisation de Tkinter et de nous avoir permis de mieux comprendre les concepts nécessaires au développement d'interfaces graphiques en Python. Les connaissances acquises durant ses cours ont constitué une base essentielle pour la réalisation de notre projet **IncidentDesk.**

I.Introduction

IncidentDesk est une application de bureau qui gère des tickets d'incidents dans une entreprise : un employé décrit un problème, l'adresse à un département, et ce département le traite en faisant évoluer son statut. Elle est écrite en Python, avec Tkinter pour l'interface et SQLite pour le stockage, et son code est réparti en trois fichiers : main.py, tickets_ui.py et database.py.

Ce compte rendu va à l'essentiel : la relation entre les trois fichiers et les classes, le rôle de chaque fichier, et les remarques qui expliquent pourquoi le code est écrit ainsi.

L'application permet de :

- se connecter avec un identifiant d'employé (par exemple R150190) ;
- créer un ticket et choisir le département destinataire ;
- consulter ses tickets et un tableau de bord avec les compteurs par statut ;
- traiter les tickets de son département en changeant leur statut (Ouvert, En traitement, Terminé, Annulé).

Quatre employés de test existent, un par département : wyam (finance), mohamed Amine (IT), Soulaimane (sales) et Prof Hain (Management). La règle principale est la suivante : un ticket n'est visible que par le département destinataire, sauf pour le manager, Prof Hain.

II. Architecture : trois fichiers, trois responsabilités

1. Organisation des fichiers

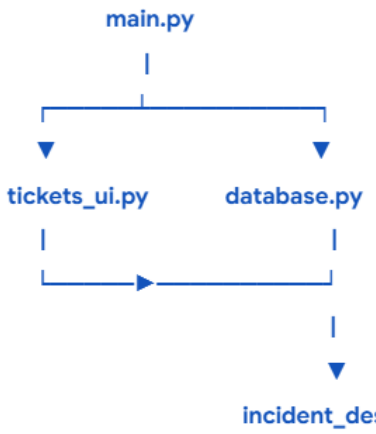
Fichier	Rôle	Contenu principal
main.py	Gestion de l'application	Classe App, connexion, navigation et tableau de bord
tickets_ui.py	Gestion de l'interface des tickets	Trois classes et cinq fonctions
database.py	Gestion des données	Création des tables et fonctions SQL

Remarques sur cette organisation :

- Les données sont isolées. database.py ne contient aucun élément d'interface. Si l'on remplaçait SQLite par une autre base, seul ce fichier changerait.

2. Relations entre les fichiers

L'organisation générale peut être représentée ainsi :



III.Présentation des fichiers:

1. database.py : gestion de la base de données:

Ce fichier représente la couche de données de l'application. Il est chargé de créer les tables, d'insérer les données initiales et de fournir les fonctions nécessaires à la manipulation des informations.

La base de données utilisée s'appelle **`incident_desk.db`**. Elle contient trois tables principales :

1.1. Table Department

Cette table contient les informations relatives aux différents départements de l'entreprise.

Champ	Description
ID	Identifiant du département, clé primaire
Name	Nom du département
Description	Description du département

1.2. Table Employee

Cette table contient les informations des employés.

Champ	Description
ID	Identifiant du de l'employé, clé primaire
Name	Nom de l'employé
DepartmentId	Identifiant du département, clé étrangère

1.3. Table Ticket

Cette table contient les tickets créés par les employés.

Champ	Description
ID	Identifiant du ticket, clé primaire
Description	Description de l'incident
CreatorID	Identifiant de l'employé créateur, clé étrangère
TargetDepartmentID	Département destinataire, clé étrangère
Status	Statut actuel du ticket

Le champ `Status` accepte quatre valeurs : `Open`, `In treatment`, `Done` et `Canceled`. Sa valeur par défaut est `Open`.

2. tickets_ui.py : gestion de l'interface des tickets

Ce fichier regroupe tout l'affichage lié aux tickets : le tableau, la page du département et la fenêtre de création. Il contient trois classes qui permettent d'organiser les différents éléments graphiques de l'application :

Classe	Hérite de	Rôle
TicketTable	ttk.Treeview	Tableau réutilisable à 5 colonnes : ID, Description, Créateur, Département, Statut
DeptPage	ttk.Frame	Page « Tickets du département » : tableau, choix du statut et bouton de changement
NewTicketWindow	tk.Toplevel	Fenêtre de création : description et département cible

Cinq fonctions font le lien avec **main.py** :

- **`clear(frame)`** : vide un cadre en supprimant tous les éléments graphiques qu'il contient. Elle permet de préparer l'affichage d'une nouvelle page.
- **`build_table(parent, rows)`** : crée un objet `TicketTable`, y insère les données reçues et affiche le tableau dans le cadre indiqué.

- **`page_mine(content, user)`** : affiche la page « Mes tickets » en récupérant uniquement les tickets créés par l'employé connecté.
- **`page_dept(content, user)`** : crée et affiche la page des tickets du département de l'utilisateur en utilisant la classe `DeptPage`.
- **`open_new_ticket(master, user, on_done)`** : ouvre la fenêtre de création d'un ticket en transmettant les informations de l'utilisateur connecté ainsi qu'une fonction de rappel exécutée après l'enregistrement.

Ainsi, les classes définissent les composants graphiques de l'application, tandis que les cinq fonctions facilitent leur utilisation depuis `main.py`. Cette organisation permet de séparer la construction des interfaces de la gestion générale de l'application.

3. main.py : gestion principale de l'application

C'est le point d'entrée : on lance l'application avec **python main.py**. La classe **App** hérite de **tk.Tk** : ce qui signifie qu'elle représente la fenêtre principale de l'application. Elle assure la gestion de la connexion, de la navigation entre les pages et de l'affichage du tableau de bord.

Ses principales méthodes sont :

- **__init__** : prépare la base (init_db, seed_initial_data) puis affiche la connexion ;
- **show_login, login, logout** : l'écran de connexion, la vérification de l'identifiant et la déconnexion ;
- **show_dashboard** : construit la barre latérale (menu, « + Nouveau ticket », « Deconnexion ») et la zone de contenu ;
- **page_home** : « Bonjour », le département, les trois cartes de comptage et les cinq derniers tickets ;
- **new_ticket** : ouvre la fenêtre de création ;
- **clear** : détruit le contenu de la fenêtre pour changer d'écran.

La classe App joue le rôle de coordinateur : elle relie les différentes interfaces aux fonctions de gestion des données et assure le déroulement général de l'application.

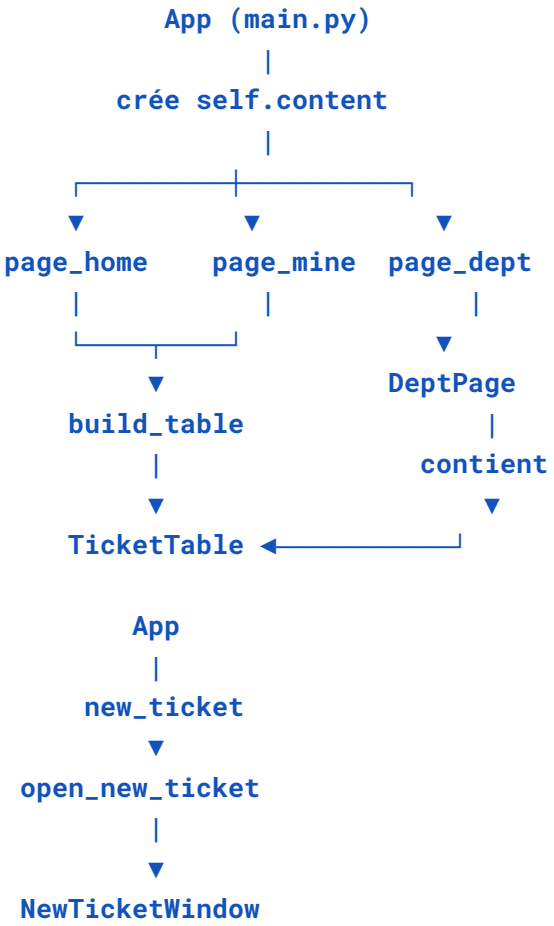
IV. Relations entre les classes

Les quatre classes du projet héritent chacune d'une classe de Tkinter ; entre elles, il n'y a pas d'héritage, seulement de la composition et des paramètres échangés.

Classe	Fichier	Hérite de	Rôle
App	main.py	tk.Tk	fenêtre principale
TicketTable	tickets_ui.py	ttk.Treeview	tableau des tickets

DeptPage	**tickets_ui.py**	ttk.Frame	page du departement
NewTicketWindow	**tickets_ui.py**	tk.Toplevel	fenêtre de création

☐ *Relations entre les classes:*



☐ *Explication des relations*

Trois mécanismes principaux assurent la communication entre les classes :

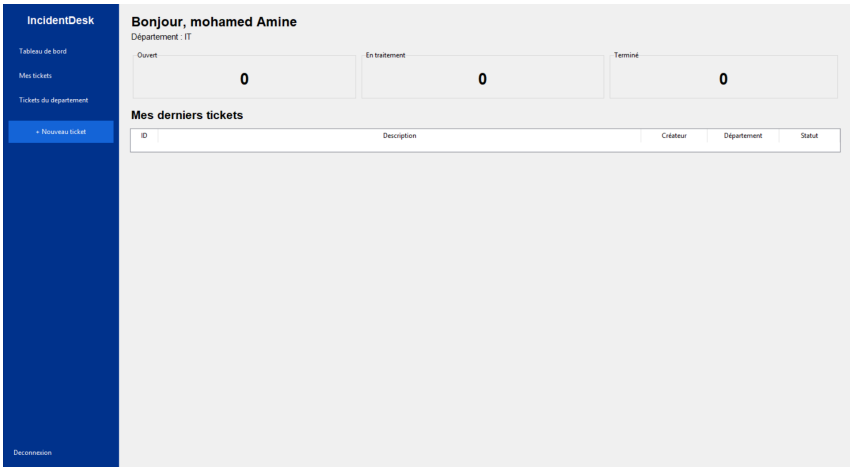
1. **La composition :** `DeptPage` contient un objet `TicketTable` pour afficher les tickets. De même, `build_table()` crée un tableau réutilisable dans les différentes pages.
2. **Le passage de paramètres :** `App` transmet `self.content` (la zone d'affichage) et `self.user` (l'employé connecté) aux fonctions et classes de `tickets_ui.py`. Cela permet aux deux fichiers de communiquer sans dépendre directement l'un de l'autre.
3. **Le callback (on_done) :** lors de la création d'un ticket, `App` transmet une fonction à `NewTicketWindow`. Après l'enregistrement, cette fonction est exécutée pour actualiser la page « Mes tickets », sans que la fenêtre ait besoin de gérer elle-même la navigation.

V. Aperçu de l'application et résultats d'exécution

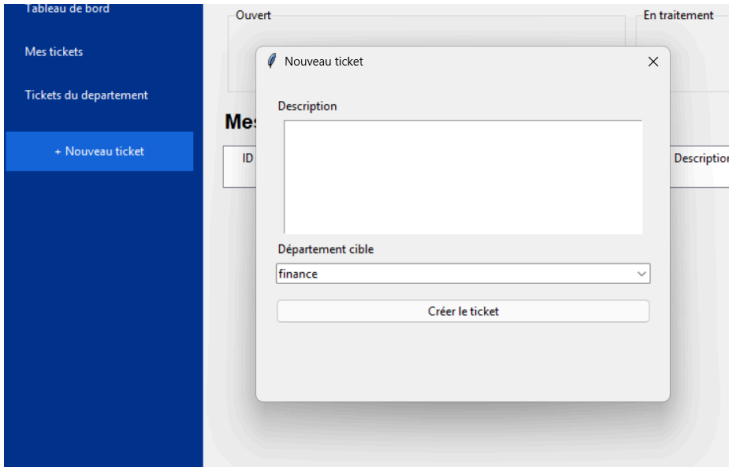
1. Écran de connexion:



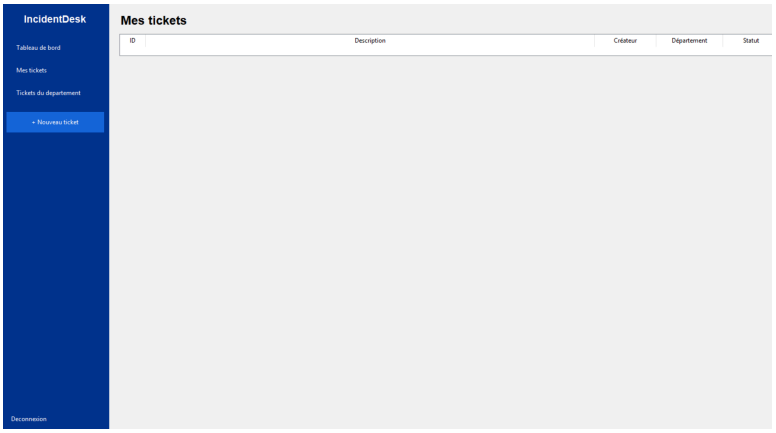
2.Tableau de bord:



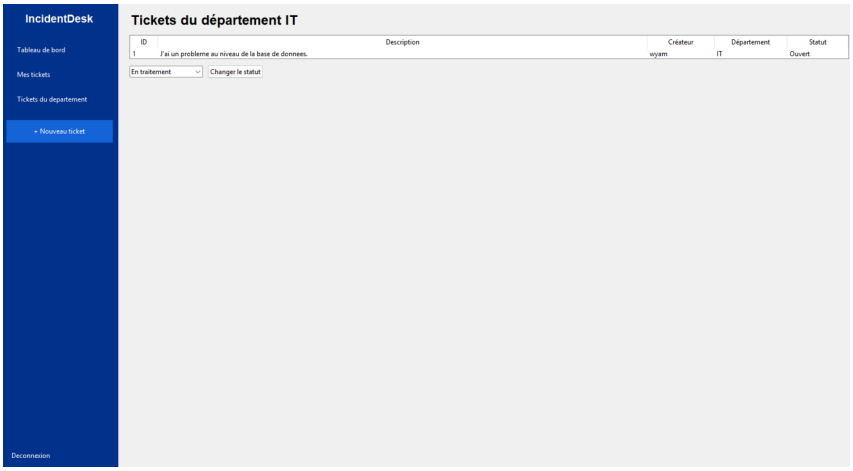
3. Création d'un ticket:



4.Mes tickets



5. Tickets du département

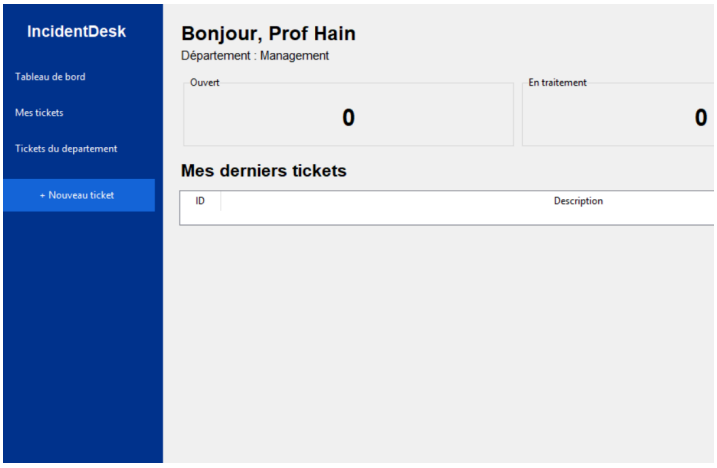


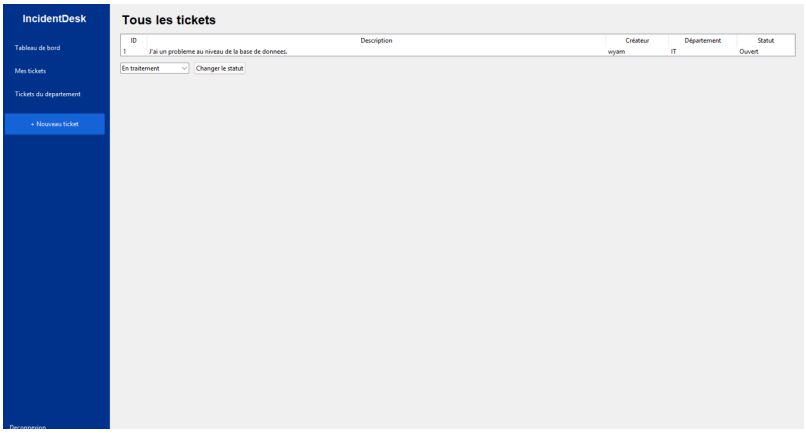
Ces captures présentent le fonctionnement de l'application à travers un exemple de connexion avec le compte de Mohamed Amine, appartenant au département informatique (IT).

- ➔ **Figure 1:**Saisie de l'identifiant de Mohamed Amine (R150191) afin d'accéder à son espace personnel.
- ➔ **Figure 2 :** Tableau de bord de Mohamed Amine: Après une connexion réussie, le tableau de bord affiche un message de bienvenue « Bonjour, Mohamed Amine », son département ainsi que les statistiques de ses tickets.
- ➔ **Figure 3 :** Cette rubrique permet à Mohamed Amine de consulter les tickets qu'il a lui-même créés. Dans cet exemple, aucun ticket n'a encore été créé par cet utilisateur ; la liste est donc vide.
- ➔ **Figure 4 :** Cette interface affiche les tickets destinés au département informatique. Un ticket est visible avec la description suivante : « J'ai un problème au niveau de la base de données ».
- ➔ **Figure 5 :** Après avoir sélectionné le ticket, son statut peut être modifié afin de suivre son traitement, par exemple de « Ouvert » à « En traitement ».

- ## La vue du Manager:

Le responsable du département Management peut consulter l'ensemble des tickets de l'entreprise, contrairement aux autres employés qui disposent d'une vue limitée à leur département. Prof Hain (R150193)





VI. LIMITES ET PERSPECTIVES D'AMÉLIORATION

1. Limites actuelles

- Connexion simplifiée, sans mot de passe.
- Rôle du manager déterminé par son département.
- Filtrage des tickets effectué en Python.
- Absence de dates de création et de modification.
- Contrôle d'accès limité, non sécurisé.

2. Améliorations possibles

- Ajouter une authentification avec mot de passe.
- Ajouter un champ `Rôle` pour identifier les managers.
- Optimiser le filtrage directement en SQL.
- Ajouter les dates de création et de modification.
- Intégrer une recherche et des filtres supplémentaires.
- Renforcer la gestion des droits d'accès.

VII.Conclusion

La réalisation du projet **IncidentDesk** nous a permis de mettre en pratique nos connaissances en Python, en programmation orientée objet, en bases de données SQLite et en interfaces graphiques avec Tkinter.

L'application permet de créer, consulter et gérer des tickets d'incidents au sein d'une entreprise. Ce projet nous a également permis de mieux comprendre l'organisation d'une application en plusieurs fichiers et l'interaction entre l'interface et la base de données.

Malgré quelques limites, **IncidentDesk** constitue une première version fonctionnelle qui pourra être améliorée et enrichie à l'avenir.